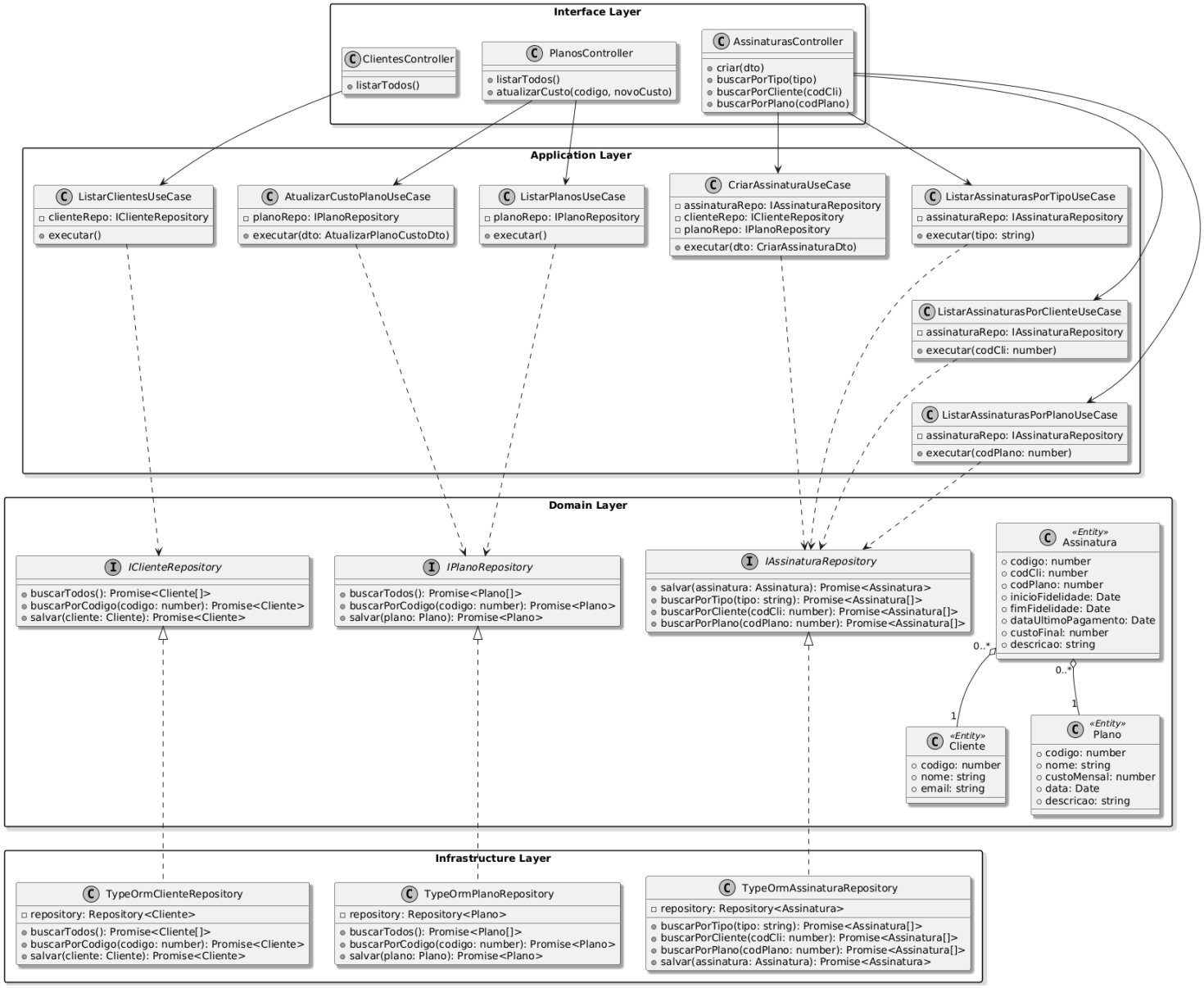
ENTREGA DO PROJETO DA DISCIPLINA
Desenvolvimento de Sistemas Backend
Fase 1

Nome do estudante: Tiago Francisco Strey

TÓPICO 1 | DIAGRAMA UML



O diagrama UML apresenta a visão estrutural do **Serviço de Gestão**, detalhando a organização modular e o relacionamento entre as classes fundamentais do sistema. A modelagem foi concebida para refletir uma arquitetura desacoplada, onde os componentes são agrupados por responsabilidade técnica e de negócio.

Destaques da Modelagem:

- **Módulos do Sistema:** O sistema é dividido em módulos lógicos que isolam as preocupações de Clientes, Planos e Assinaturas. Cada módulo é composto por seus respectivos controladores, serviços de aplicação e repositórios.
- **Entidades de Domínio:** As classes Cliente, Plano e Assinatura possuem atributos e comportamentos que refletem as regras essenciais da operadora de internet, como a gestão de períodos de fidelidade e custos mensais.
- **Relacionamentos:** O diagrama evidencia a associação entre as classes. Uma Assinatura atua como o elo de ligação, possuindo referências obrigatórias a um Cliente e a um Plano, garantindo a integridade referencial do modelo de negócio.
- **Abstrações (Interfaces):** São apresentadas as interfaces de repositório (ex: IPlanoRepository), que servem como contratos para a camada de infraestrutura. Isso demonstra visualmente como a aplicação se comunica com o banco de dados sem depender de uma implementação específica.

TÓPICO 2 | ORGANIZAÇÃO DAS CLASSES e PRINCÍPIOS SOLID

A arquitetura do **Serviço de Gestão** foi estruturada para garantir o isolamento das regras de negócio e a facilidade de manutenção, aplicando rigorosamente os cinco princípios SOLID:

- **S - Single Responsibility Principle (SRP):** As responsabilidades estão estritamente separadas. Os Controllers (camada Interface) lidam apenas com requisições HTTP. Os Use Cases (camada Application) executam exclusivamente as lógicas de negócio. As Entities (camada Domain) contêm apenas o estado e a validação primária dos dados.
- **O - Open/Closed Principle (OCP):** O sistema permite a adição de novas funcionalidades (ex: novos casos de uso para relatórios) sem a necessidade de alterar o código de infraestrutura ou as entidades centrais já validadas.
- **L - Liskov Substitution Principle (LSP):** As classes concretas da infraestrutura (como ClienteRepositoryTypeORM) implementam contratos definidos no domínio. Elas podem ser substituídas por outras implementações (ex: um repositório em memória para testes) sem quebrar o funcionamento da aplicação.
- **I - Interface Segregation Principle (ISP):** Foram criadas interfaces de repositório granulares e específicas para cada contexto (IClienteRepository, IPlanoRepository, IAssinaturaRepository), evitando que os serviços dependam de métodos genéricos que não utilizam.
- **D - Dependency Inversion Principle (DIP):** É o pilar da separação de camadas. A camada de Application não depende da Infrastructure (TypeORM/SQLite). Ela interage apenas com as abstrações (Interfaces) definidas no Domain. O framework NestJS injeta as implementações reais em tempo de execução.

TÓPICO 3 | ARQUITETURA LIMPA

A implementação do Serviço de Gestão adota padrões de projeto consolidados para sustentar as premissas da Arquitetura Limpa e garantir o isolamento das camadas:

- **Repository Pattern (Padrão Repositório):** Isola as regras de domínio dos detalhes de persistência de dados. A camada de Infrastructure utiliza o TypeORM para acessar o banco de dados (SQLite), mas o faz implementando estritamente os contratos (interfaces) definidos na camada de Domain.

- **Dependency Injection (Injeção de Dependência):** Gerenciada nativamente pelo framework NestJS. Permite que as implementações concretas dos repositórios sejam injetadas nos Use Cases (camada Application) em tempo de execução, promovendo o baixo acoplamento estrutural.
- **Data Transfer Object (DTO):** Padrão aplicado na camada de Interface. Os DTOs definem o formato dos dados de entrada (ex: CriarAssinaturaDto) e utilizam a biblioteca class-validator para barrar requisições malformadas antes que atinjam a lógica de negócio.
- **Regra de Dependência:** O fluxo de acoplamento do sistema aponta exclusivamente de fora para dentro. A camada central (Domain) é completamente "limpa" e não importa bibliotecas externas. A camada Application orquestra o fluxo dependendo apenas do Domain.
- **Independência de Frameworks e Banco de Dados:** Graças a essa arquitetura, as regras de negócio centrais (Entidades e Casos de Uso) desconhecem a existência do NestJS, do SQLite ou do protocolo HTTP, o que blinda o núcleo do sistema contra mudanças tecnológicas.

TÓPICO 4 | ESTRUTURA

O projeto foi empacotado e organizado para facilitar a avaliação, a instalação das dependências e a execução imediata. O arquivo .zip entregue contém a seguinte estrutura essencial:

- **Código-Fonte (src/):** Diretório principal contendo toda a implementação do Serviço de Gestão em TypeScript, rigorosamente separada nas pastas domain, application, infrastructure e interface, refletindo a Arquitetura Limpa.
- **Gerenciador de Dependências (package.json):** Arquivo de configuração do Node.js/NestJS que lista as bibliotecas utilizadas (TypeORM, SQLite3, class-validator) e define os scripts de execução (ex: npm run start).
- **Instruções de Execução (README.md):** Documentação técnica completa com o passo a passo para subir o servidor, detalhando requisitos, comandos de terminal e regras de negócio.
- **Coleção de Testes (.json):** Arquivo da ferramenta Postman anexado na raiz do projeto, contendo todas as requisições HTTP pré-configuradas (GET, POST, PATCH) para validar os endpoints e as regras de negócio construídas.
- **Banco de Dados (SQLite):** O sistema utiliza o SQLite em arquivo local. Caso o arquivo database.sqlite não seja enviado no .zip por restrições de versionamento, a entidade SeedService (implementada na infraestrutura) encarrega-se de criar o banco e popular os dados iniciais automaticamente na primeira execução do servidor NestJS.

TÓPICO 5 | CONCLUSÃO

O desenvolvimento da Fase 1 do Serviço de Gestão atingiu os objetivos propostos e representou um grande aprendizado prático sobre a Arquitetura Limpa.Durante a implementação, um dos principais desafios iniciais foi assimilar a rigorosa separação de responsabilidades e resistir à tentação de acessar o banco de dados diretamente pelos Controllers. A solução foi manter a disciplina de criar Casos de Uso (Use Cases) até mesmo para rotas simples de listagem, consolidando o entendimento prático de que essa camada intermediária é vital para isolar e proteger as regras de negócio. Outro desafio comum superado foi a diferenciação estrutural entre os objetos de transferência (DTOs), as entidades puras do domínio e os modelos de persistência. Para resolver esse conflito conceitual, padronizamos um fluxo onde o class-validator atua estritamente nos DTOs na entrada da requisição, enquanto as interfaces de repositório garantem que a camada de Aplicação manipule apenas Entidades puras, delegando os detalhes de tabelas e do TypeORM exclusivamente para a camada de infraestrutura.